COMPUTER GRAPHICS | GAME ENGINE ENGINEERING

Adaptive Level of Detail for Real-Time Rendering in Godot

A feedback-controlled, distance-based LOD for vegetation, compared against a fixed-threshold baseline

Shreeniwas Salian

Problem Dense vegetation scenes cost frame time; fixed LOD distances cannot react when the GPU is overloaded, so detail is either wasted or cut blindly.



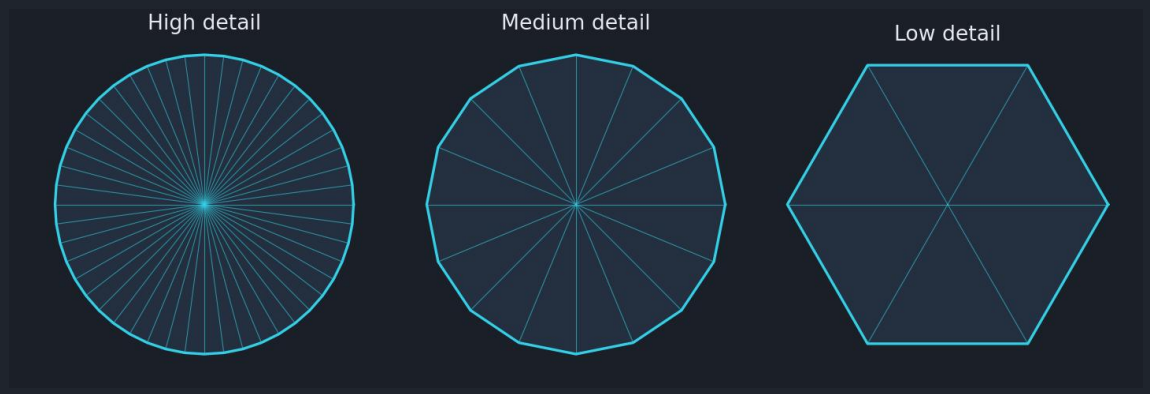
Real capture from the project (Godot 4.7.2, Terrain3D 1.0.2, 1920x1080, baseline LOD, fixed probe view at path start).

41,698
vegetation instances

528 m
fixed camera path

3 modes
none / baseline / adaptive

Why LOD, and where fixed thresholds fall short



Schematic only: polygon count falls as distance grows (not the project meshes).

LOD0	LOD1	LOD2
0 to 60 m	60 to 150 m	beyond 150 m

Baseline tree thresholds from the code (60 m / 150 m, 10 m hysteresis).

General LOD concepts

- Cost scales with triangles, draw calls and overdraw
- Distant objects need fewer triangles for the same look
- Fixed distances ignore the actual frame budget

Implemented in this project

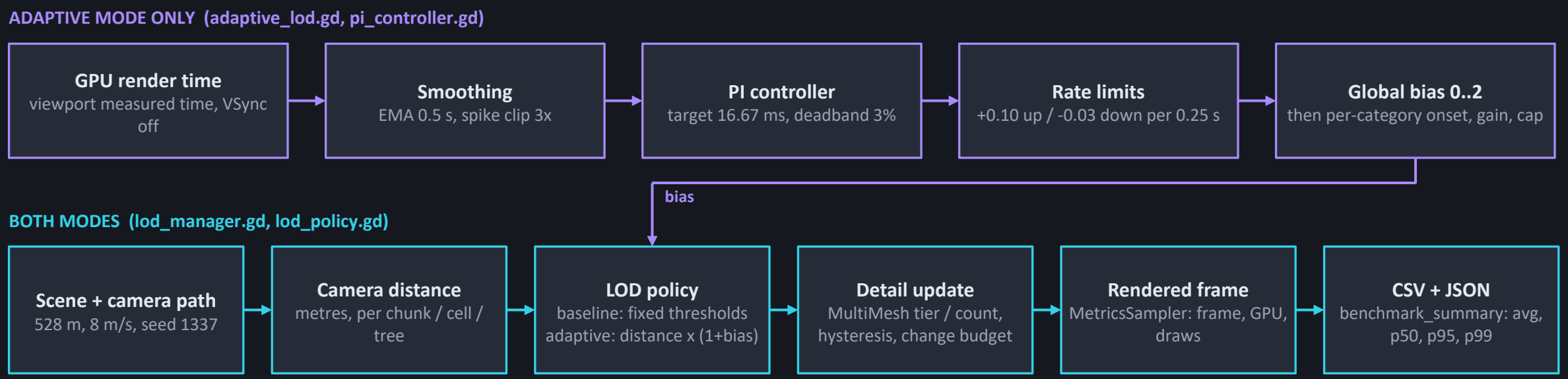
- Fixed-threshold baseline for 4 categories: trees (3 mesh levels), grass (3 tiers + cull), bushes and debris (density thinning)
- Adaptive mode: one PI-controlled bias scales the same thresholds from a measured GPU render time
- Hysteresis, rate limits and a per-update change budget to limit popping
- Same scene, seed and camera path for every mode

Not implemented

- Per-instance or screen-size importance
- Cross-fade or dither between mesh levels (trees still pop)
- Mesh LOD for bushes (density only)

SYSTEM ARCHITECTURE

Architecture: one controller bias scales fixed metre thresholds



Thresholds in metres

- Trees: effective = max(floor, base / (1 + bias)), never above base
- Hysteresis added in metres (10 m)
- Bias 0 gives exactly the baseline, so the same code serves both modes

Bounded coarsening

- Tree floors: LOD1 never before 60 m, LOD2 never before 100 m
- Per-category caps: grass 1.0, trees 1.0, bushes 0.5, debris 0.5
- bias_min 0: never finer than baseline. bias_max 2.0

baseline_lod.gd: _rebuild_thresholds()

```
for i in 2:
  var effective: float = base[i] /
_dist_scale
  if _dist_scale > 1.0:  # coarser
    effective = minf(base[i],
      maxf(effective, floors[i]))
  _thresholds[i] = effective
```

NOVELTY

Technical contributions, and what the evidence supports

Contribution	Baseline does	Adaptive changes	Why it matters	Evidence	Status
1 Adaptive thresholds	Fixed distances, 60 / 150 m for trees	One PI-controlled bias scales all thresholds from GPU time	Reacts to load instead of ignoring it	Controller self-tests (documented passing); bias saturates at 2.0 in stress	**Implemented**
2 Real-world units	Metres, fixed	Floors, caps and hysteresis in metres; coarser = distance x (1+bias)	Visible effect is predictable and tunable in scene units	Code + tests: tree LOD never differs from baseline within 100 m	**Implemented**
3 Constrained coarsening	No notion of a limit	Tree floors 60 / 100 m, category caps, bias range 0..2	Keeps nearby detail intact	0 tree violations within 100 m at bias 2.0 (self-test). Not literally a 2-level limit	**Partly as stated**
4 Unified benchmark	n/a	Same scene, seed, path (sha256 logged), interleaved runs, CSV + JSON	Fair, repeatable comparison	11 runs in the m4 series; hardware state drifted between some runs	**Implemented, noisy**
5 Quality / speed trade-off	Full detail baseline	Measures primitives, draws, GPU ms and image difference	Shows what a saving costs	Primitives -5 to -10% (fixed view). GPU gain only in stress: -5.5%. No perceptual metric	**Partly demonstrated**

Scope of the claim: these are this project's technical contributions. No literature review was done, so none is claimed as globally novel; feedback-controlled quality scaling is an established idea in general.

METHODOLOGY

Experimental setup and fairness of the comparison

Condition	Baseline	Adaptive
Software	Godot 4.7.2-stable, DEBUG + EDITOR build, Windows	same
Hardware	Intel i7-8850H (12 threads), NVIDIA Quadro P3200 laptop GPU	same machine
Renderer	Forward+, D3D12, no MSAA / TAA, VSync off, FPS uncapped	same
Resolution	Normal 1920x1080 | Stress 2560x1440	same
Scene	Terrain3D 1.0.2, 41,698 instances + 403k grass clumps, seed 1337	same
Camera	528 m path, 8 m/s, 10 s warm-up, 60 s measured	same, controller reset to bias 0 at start
LOD	Fixed thresholds, 4 categories	Same thresholds x PI bias; target 16.67 ms (normal), 8 ms (stress)
Runs	Normal 3, stress 2	Normal 3, stress 2 (+1 reachable-target run)
Metrics	Per-frame frame time, GPU ms, draws, primitives, switches, policy CPU ms (CSV, ~2k to 6k rows per run)	+ bias, controller terms

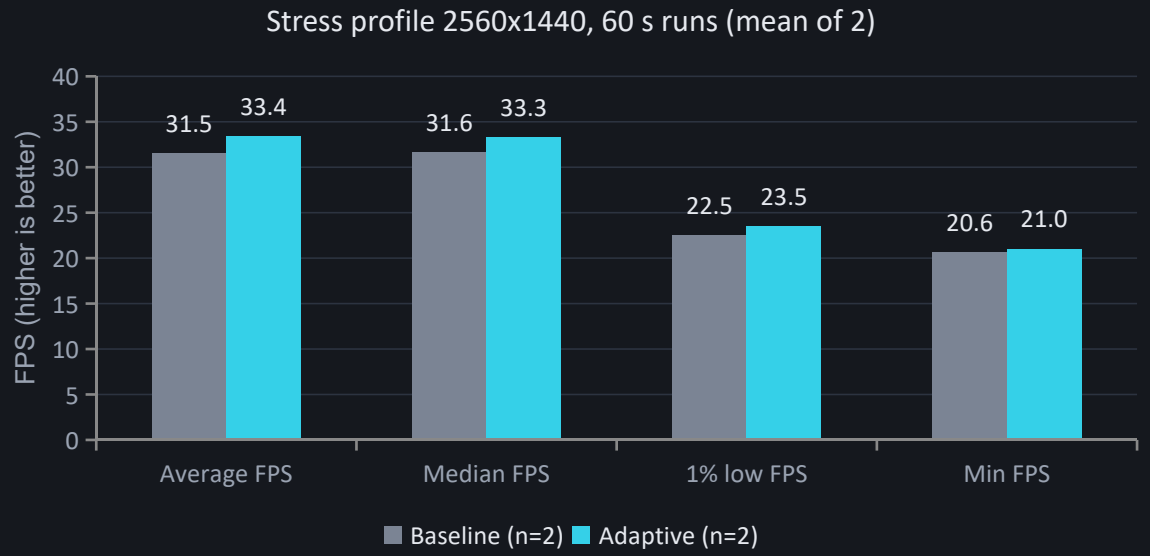
Fairness risks found

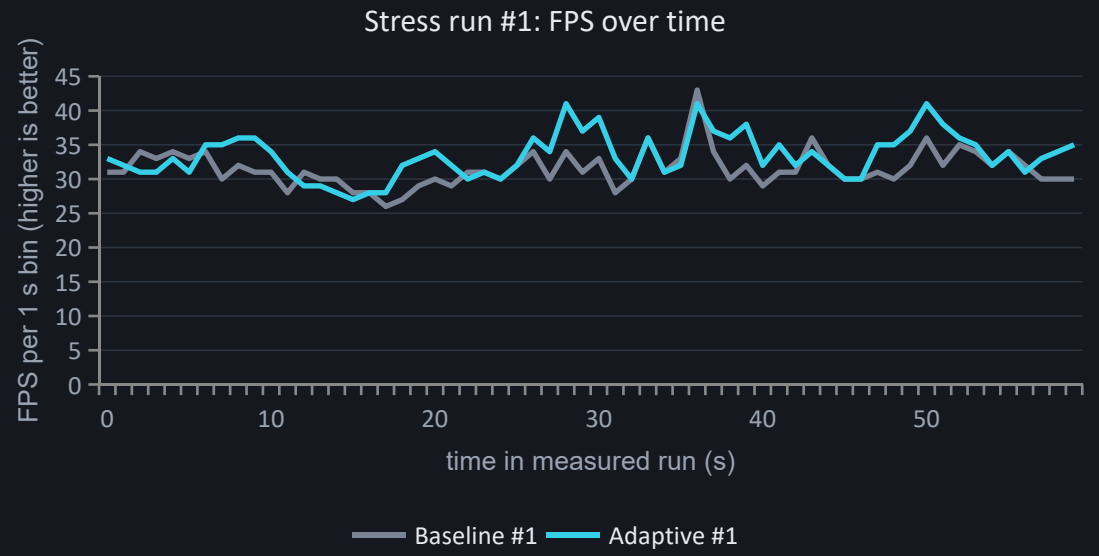
- Runs alternate baseline / adaptive, but GPU speed drifted: normal baseline #1 ran at 9.3 ms GPU, #2 and #3 at ~19 ms
- Debug + editor build, so CPU timings are inflated
- Results used the milestone-4 mapping; the current, gentler mapping has no timed run

Missing, please supply

- GPU driver version, RAM, power plan, thermals
- Release / export-build run
- More repeats (2 to 3 per cell is thin)
- Idle machine, editor closed

Results: a small gain under stress, no gain at 1080p





Metric	Stress base	Stress adapt	Change	Normal base*	Normal adapt	Change
Average FPS	31.5	33.4	+5.8%	47.3	46.6	-1.5%
Median FPS	31.6	33.3	+5.5%	48.3	46.8	-3.1%
1% low FPS	22.5	23.5	+4.2%	29.4	31.2	+6.2%
Min FPS	20.6	21.0	+2.1%	22.7	27.5	+21.5%
Mean frame ms (lower = better)	31.7	30.0	-5.5%	21.1	21.5	+1.4%

*Normal baseline = runs #2 and #3. Including run #1 (104 FPS avg, GPU state changed) gives 66.2 vs 46.6 FPS avg, -29.6% for adaptive; that run is shown, not hidden, but is not a like-for-like pair. Change = (Adaptive - Baseline) / Baseline x 100. Normal n = 3 adapt vs 2 base.

Sub-10 FPS check

- 0 frames under 10 FPS in all 6 adaptive 60 s runs; worst frame 20.7 FPS (48 ms)
- One frame at 3.7 FPS: baseline #1 (not adaptive)
- Not logged: your live drops. Hypothesis only: the ~0.7 to 1 s first-enable build when switching mode

Visual comparison: baseline vs adaptive at the same view



Baseline, full view



Adaptive (bias 2.0), full view



Baseline, crop of left treeline and mid grass



Adaptive, same crop

What changes

- Near field is identical: nearby trees keep baseline LOD (floors)
- Mid-field grass is thinner; distant bushes sparser
- Main-pass primitives -9% at this view (-5% at 150 m, -10% at 300 m)
- Mean abs image difference 4.27 / 0.87 / 0.98 (0 / 150 / 300 m)

Fixed-view probe with bias pinned at 2.0, not a live controller run. Tree mesh swaps still pop (no cross-fade).

VIDEO PLACEHOLDER

No screen recording or URL exists in the project folder. In PowerPoint: Insert > Video > This Device, then choose your Godot capture (baseline vs adaptive, same path).

SUMMARY

Conclusion, limitations and next steps

Implemented

- Baseline and adaptive LOD for 4 vegetation categories
- PI controller with rate limits, deadband, anti-windup
- Metre-based floors keep near-field detail
- Reproducible benchmark with CSV + JSON

What the data shows

- Stress 1440p: +5.8% avg FPS, -5.5% frame time (n=2)
- 1080p: no measurable change (-1.5% avg FPS)
- Visual change small and localised, primitives -5 to -10%
- Targets (16.67 / 8 ms) never reached

Limitations

- Few runs, drifting GPU state, debug build
- No timed run of the current mapping
- Live <10 FPS drops not captured in logs
- No perceptual metric, tree pops, no importance

Future work

Tune floors and caps, add cross-fade to remove pops, profile with release builds and a GPU profiler, move the first-enable build off the live frame, add a CPU-bound signal, repeat on an idle machine with more runs, add SSIM or FLIP.

Takeaway The adaptive mechanism is implemented and bounded, and it saves a few percent of GPU time only when the GPU is heavily overloaded. Visual quality is largely preserved. Improved performance is not yet demonstrated in the normal profile.